OS Lab Assignment 8 — Memory Management in xv6

Building and Running

```
make qemu-nox CPUS=1     # used for the per-question logs
make qemu-nox            # 2 CPUs (full_session_2cpu.txt)
```

Inside xv6, run `vatest`, `memtest`, `vmprinttest` and `lazytest`. The terminal output of every run is in `output_logs/`.

Folder	Contents
`kernel/`	`vm.c`, `trap.c`, `kalloc.c`, `proc.c`, `proc.h`, `exec.c`, `sysproc.c`, `syscall.c`, `syscall.h`, `defs.h`, `meminfo.h` (new)
`user/`	`vatest.c`, `memtest.c`, `vmprinttest.c`, `lazytest.c`, `user.h`, `usys.S`
`Makefile`	adds the four programs to `UPROGS`, the `-smp N,sockets=N` fix, and drops `-Werror`
`output_logs/`	per-question terminal logs and the full 2-CPU session

New system calls: `va2pa` (22), `getmeminfo` (23) and `vmprint` (24). All four questions are built on a clean copy of xv6-public. Earlier assignments' changes are not included, so the A5 shared-memory page at `0x60000000` doesn't show up in the page-table dumps.

The assignment's submission list also names `cowtest.c`, `protecttest.c`, `shmtest.c` and `pagerepl.c` (Q5–Q8). The assignment PDF only contains Q1–Q4, so only those four are implemented.

Q1 — Virtual to physical address translation (`va2pa`)

Design. `walkpgdir()` in `vm.c` was made non-static and declared in `defs.h`. `sys_va2pa` reads `va` with `argint` and calls `walkpgdir(myproc()->pgdir, va, 0)`, with `alloc = 0` so it never creates a page table. If the PDE/PTE is missing, `PTE_P` is clear, or `PTE_U` is clear, it returns `-1`. Otherwise it returns `PTE_ADDR(*pte) | (va & 0xFFF)`. It also prints `PDX(va)`, `PTX(va)`, the offset and the P/W/U bits with `cprintf` (lines tagged `[kernel]`).

Result (CPUS=1):

Variable	VA	PDX / PTX / offset	PA
global	`0xD78`	0 / 0 / `0xd78`	`0xDEE8D78`
local (stack)	`0x2FCC`	0 / 2 / `0xfcc`	`0xDEE5FCC`
`malloc(100)`	`0xAF98`	0 / 10 / `0xf98`	`0xDF76F98`
`0x7FFFFFFF`	—	511 / 1023 / `0xfff`	**-1** (not mapped)
`0x80100000` (kernel)	—	512 / 256 / `0x0`	**-1** (PTE_U clear)

After `fork()`, both processes report `&global = 0xD78`, but the child's PA is `0xDF2CD78` and the parent's is `0xDEE8D78`. `copyuvm()` gives the child its own copy of every frame, so the same VA maps to different PAs.

Q2 — Physical memory statistics (`getmeminfo`)

Design and data structures. `kmem` in `kalloc.c` gained `int nfree` and `int ntotal`. `kfree()` does `nfree++` and `kalloc()` does `nfree--` (only on success), both inside the `kmem.lock` critical section. `freerange()` counts `ntotal` during boot. `kmemstat()` reads both values under the lock.

```
struct meminfo { int total_pages; int free_pages; int proc_pages; int pt_pages; };   /* meminfo.h */
```

`proc_pages = sz / PGSIZE`. `pt_pages` comes from `ptpages()` in `vm.c`, which counts 1 for the page directory plus one per present PDE. That includes the kernel-half page tables each process gets from `setupkvm()`, so the number is the real page-table cost of the process (66 for a small program, 64 of them for the kernel mappings).

Memory before/after (CPUS=1):

Point in `memtest`	free_pages	proc_pages	Change
start	56,789	3	—
after `sbrk(16 pages)` + `memset`	56,773	19	−16
inside forked child	56,687	19	−86 (16+3 user copy, 66 page tables, 1 kstack)
after child exits and is reaped	56,773	19	back to the pre-fork value → **no leak**
after `sbrk(-16 pages)`	56,789	3	back to the start value

Q3 — Dumping the page table (`vmprint`)

Design. `vmprint(pgdir, sz)` in `vm.c` scans all 1024 PDEs. PDEs at or above `PDX(KERNBASE)` are only counted. For every other present PDE it prints the index, the page table's PA and its flags. It then walks the page table (`P2V(PTE_ADDR(pde))`) and prints each present PTE with its index, `PGADDR(i, j, 0)`, the frame's PA and its flags, indented one more level. A PTE with `P` set but `U` clear is marked `<-- guard page`. The summary line compares the number of user pages with `sz/PGSIZE − guard pages`.

Result (`vmprinttest`):

Call	sz	Entries	User pages	Guard	Check
start-up	`0x3000`	pte 0 (code/data), pte 1 (guard, `PW-`), pte 2 (stack)	2	1	3 − 1 = 2 ✓
after `sbrk(8*4096)`	`0xB000`	+ pte 3 … 10	10	1	11 − 1 = 10 ✓
after `sbrk(-4096)`	`0xA000`	pte 10 gone	9	1	10 − 1 = 9 ✓

In every dump, 64 kernel PDEs are summarized on one line. The guard page is the page `exec()` cleared with `clearpteu()`, at VA `0x1000` just below the stack. Because of Q4, the heap pages only appear once they are touched, so `vmprinttest` writes one byte to each new page before calling `vmprint()`.

Q4 — Lazy allocation of the heap

Design and data structures.

- `sys_sbrk`: for `n > 0` it only does `myproc()->sz += n`, after rejecting overflow and `sz + n >= KERNBASE`. For `n < 0` it still calls `growproc()`, which calls `deallocuvm()`. That function already skips non-present PTEs, so only frames that were really allocated get freed.
- `struct proc` gained `uint heapbase`. `exec()` sets it to the top of the user stack, which is where the heap starts, and `fork()` copies it.
- `trap.c`, `case T_PGFLT` → `lazyfault()`. It reads `rcr2()` and accepts the fault only when `heapbase <= va < sz` and the PTE isn't present. In that case it does `kalloc()`, then `memset(0)`, then `mappages(...,`

`PTE_W|PTE_U)` (`mappages` was made non-static), and the faulting instruction is restarted. Every other case prints a reason (beyond sz / below the heap / protection fault / out of memory) and falls through to the `default:` handler. That handler kills a user process, or panics on a real kernel bug.

- Kernel accesses to lazy memory: `read()`/`write()` buffers are touched by the kernel through the user's page table. Those faults arrive with `cs` in kernel mode and go through the same `lazyfault()`, so the system call just continues.
- `copyuvm()` in `fork()` now skips (`continue`s) both a missing page table and a non-present PTE instead of panicking.

Memory before/after (CPUS=1, `lazytest`):

Step	free_pages	proc_pages	Frames used
before `sbrk(32 MB)`	56,788	4	—
after `sbrk(32 MB)`	56,788	8,196	**0**
touch 4 pages (0, 8, 16, 32 MB−1)	56,781	8,196	7 (4 data + 3 new page tables)
`read()` / pipe `write()` / `read()` with untouched buffers	—	—	6 (3 data + 3 page tables), the syscalls succeed
fork, child touches 1 page, exits, reaped	56,775 → 56,775	—	**no leak**
child `sbrk(400 MB)` and touches every page	56,775 → 56,775	—	child killed on `kalloc()` failure, **no leak**
`sbrk(-32 MB)`	56,782	4	data frames freed. The 6 page tables are kept until exit (as in stock xv6)

Invalid accesses each kill only the child, and the parent and kernel keep running. The cases tested are an address beyond `sz` (`0x2007000`), the stack guard page (`0x2000`, err 7 = protection violation), a kernel address (`0x80100000`), and running out of memory. The touched pages read `0` before being written, which confirms they are zero-filled.

Problems faced and how they were resolved

1. **`pte_t` in `defs.h`.** `defs.h` is included before `mmu.h` in most files, so declaring `walkpgdir` as returning `pte_t*` broke the build. It is declared as `uint*` instead, which is the same type.
2. **`walkpgdir` creating page tables.** `va2pa`, `vmprint` and `lazyfault` all call it with `alloc = 0`, so looking up an address never allocates memory.
3. **`copyuvm` panics with a lazy heap.** Forking a process with an untouched 32 MB heap hit `panic("copyuvm: pte should exist")`. Both checks now `continue`.
4. **Faults raised inside the kernel.** `read()` into an untouched buffer faults in kernel mode, which the stock code treats as a kernel bug and panics. `T_PGFLT` is now checked against the process's heap range before the "kernel mode → panic" path, so these faults are served transparently.
5. **Q3 vs Q4 interaction.** With lazy allocation, `sbrk(8*4096)` alone adds no PTEs. `vmprinttest` touches each new page, so the dump shows them and the `sz/PGSIZE − guard` check holds.
6. **Counting page tables.** Each process has its own copy of the kernel page tables (64 of them), so `pt_pages` and the cost of a fork are much larger than the user part alone. That is why one fork costs 86 frames.
7. **Limitation.** If `kalloc()` fails while the *kernel* is faulting on a lazy page (OOM inside a system call), the process is marked killed but the kernel still panics, because the faulting kernel instruction can't be resumed. User-mode OOM, as tested, is handled cleanly.